

Week 13 – HPC Pitfalls

02613 Python and High-Performance Computing

HPC Pitfalls

Common errors that HPC users face

HPC Pitfalls

Common errors that HPC users face – but we won't!

- Some issues affect only you
 - Some issues affect the system – i.e, everyone at DTU
- 
- so also you!
- Some issues are specific to DTU HPC
 - Some issues are general for Linux systems
 - Some issues are general for "cloud systems"

HPC Pitfalls

"[...]"

- *Why do you use 'tee ...' in your job script? That replicates the output into the stdout file of LSF, which means you use double the disk space! [...]"*

– Real mail I got from HPC support after asking for advice

"[...] I don't know why 'tee' is used to be honest. It's not my software [...]"

– My response

It's not my code

It's not my Python package

HPC Pitfalls

"[...]"

- *Why do you use 'tee ...' in your job script? That replicates the output into the stdout file of LSF, which means you use double the disk space! [...]*

- Real mail I got from HPC

**Moral: your code may be fine,
but someone else's may not!**

my response

It's not my code

It's not my Python package

Pitfall 1: excessive I/O

```
#!/bin/bash
#BSUB -J name
#BSUB -q hpc
#BSUB -W 2
#BSUB -R "rusage[mem=512MB]"
#BSUB -n 4
#BSUB -R "span[hosts=1]"
#BSUB -o name_%J.out          Output file (stdout)
#BSUB -e name_%J.err          Output file (stderr)

source /dtu/projects/02613_2024/conda/conda_init.sh
conda activate 02613

python -u script.py input.npy 4
```

Issue:

For *lots or and/or frequent* output
-o/-e will be slow.

Pitfall 1: excessive I/O

```
#!/bin/bash
#BSUB -J name
#BSUB -q hpc
#BSUB -W 2
#BSUB -R "rusage[mem=512MB]"
#BSUB -n 4
#BSUB -R "span[hosts=1]"
#BSUB -o name_%J.out           Output file (stdout)
#BSUB -e name_%J.err           Output file (stderr)

source /dtu/projects/02613_2024/conda/conda_init.sh
conda activate 02613

python -u script.py input.npy 4 \
    1> output_${LSB_JOBID}.txt \  Redirect stdout to output_12345678.txt
    2> error_${LSB_JOBID}.txt     Redirect stderr to error_12345678.txt
```

Issue:

For *lots or and/or frequent* output
-o/-e will be slow.

Solution:

Manually redirect output to files.
Avoids slower -o/-e channels.

Pitfall 1: excessive I/O

Using -o/-e channels

```
#!/ Successfully completed.  
#BS  
#BS Resource usage summary:  
#BS  
#BS CPU time :
```

Manual redirect

Successfully completed.

Resource usage summary:

*"[...] I don't know why 'tee' is used to be honest.
It's not my software [...]"*

		305.00 MB
	Max Swap :	-
source	Max Processes :	4
code	Max Threads :	5
	Run time :	96 sec.
python	Turnaround time :	97 sec.

```
Delta Memory : -
Max Swap : -
Max Processes : -
Max Threads : -
Run time : 16 sec.
Turnaround time : 6 sec.
```

Pitfall 1: excessive I/O

```
#!/bin/bash
#BSUB -J name
#BSUB -q hpc
#BSUB -W 2
#BSUB -R "rusage[mem=512MB]"
#BSUB -n 4
#BSUB -R "span[hosts=1]"
#BSUB -o name_%J.out           Output file (stdout)
#BSUB -e name_%J.err           Output file (stderr)

source /dtu/projects/02613_2024/conda/conda_init.sh
conda activate 02613

python -u script.py input.npy 4 \
    1> output_${LSB_JOBID}.txt \
    2> error_${LSB_JOBID}.txt
```

Redirect stdout to output_12345678.txt

Redirect stderr to error_12345678.txt

Issue:

For *lots or and/or frequent* output
-o/-e will be slow.

Solution:

Manually redirect output to files.
Avoids slower -o/-e channels.

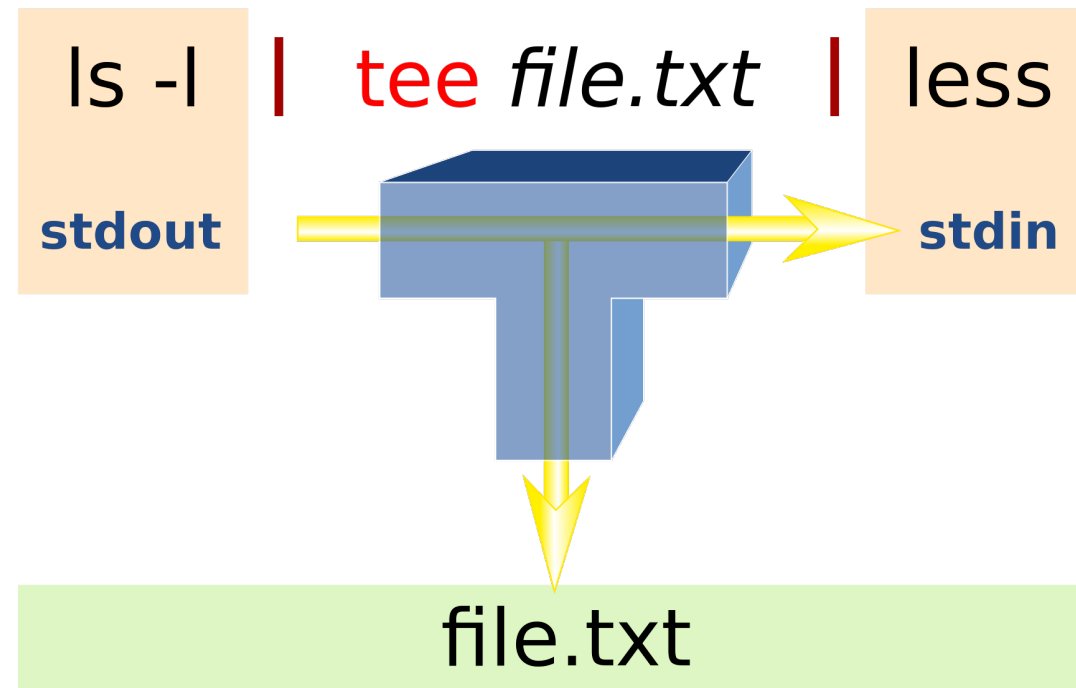
Caveat: depends on file system!
DTU HPC: work3
not the home directories

Pitfall 2: duplicate I/O

“tee: don't use tee in batch jobs!

Doesn't make sense, but duplicates amount of data!”

- HPC Support



Wikipedia user Sven, CC BY-SA 4.0

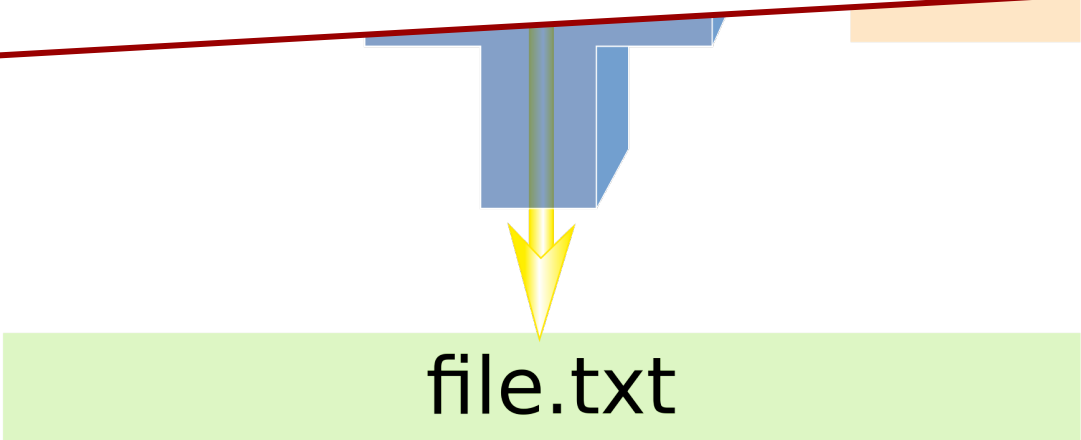
Pitfall 2: duplicate I/O

“tee: don't use tee in batch jobs!

Doesn't make sense, but duplicates amount of data!”

- HPC Support

**Moral: either print or log to file/web/other,
but *not* both!**



file.txt

Wikipedia user Sven, CC BY-SA 4.0

Pitfall 3: spamming bjobs/bstat

```
$ bsub < submit.sh
Job <702576> is submitted to queue <hpc>.
$ bstat
JOBID  USER  QUEUE  JOB_NAME  NALLOC  STAT  ...
702576 ...   hpc    name           4  PEND  ...
$ bstat
JOBID  USER  QUEUE  JOB_NAME  NALLOC  STAT  ...
702576 ...   hpc    name           4  PEND  ...
$ bstat
JOBID  USER  QUEUE  JOB_NAME  NALLOC  STAT  ...
702576 ...   hpc    name           4  PEND  ...
$ bstat
JOBID  USER  QUEUE  JOB_NAME  NALLOC  STAT  ...
702576 ...   hpc    name           4  PEND  ...
$ bstat
JOBID  USER  QUEUE  JOB_NAME  NALLOC  STAT  ...
702576 ...   hpc    name           4  RUN   ...
```

Pitfall 3: spamming bjobs/bstat

```
$ bsub < submit.sh  
Job <702576> is submitted to queue <hpc>.  
$ watch bstat
```



Automatically call 'bstat' every 2 seconds

Pitfall 3: spamming bjobs/bstat

```
$ bsub < submit.sh  
Job <702576> is submitted to queue <hpc>.  
$ watch bstat
```

Automatically call 'bstat' every 2 seconds



Issue:

Calling bstat or bjobs interrupts the scheduler!
Bad for all of us!

Also, data is only updated 1-2 times a minute

Solution:

Patience!

For automation: wait 30 seconds between calls

```
watch -n30 bstat
```

Better: watch -n60 bstat

Best: ask yourself, "do I really need this?"

Totally insane: watch -n 0.1 bstat

Pitfall 4: too many files in one folder

```
$ ls bigdata | wc -l
100000
$ time ls bigdata
...
real    0m2.635s
user    0m0.408s
sys     0m0.208s
$ ls bigdata/*.jpg
bash: /usr/bin/ls: Argument list too long
```



bigdata has
100K files

Issue:

Thousands of files in one folder = slowdowns

Solution:

Make subfolders that each store 1000 files

Pitfall 4: too many files in one folder

```
$ ls bigdata | wc -l
100000
$ time ls bigdata
...
real    0m2.635s
```

← bigdata has 100K files

Remember: job array exercise

```
$ ls /dtu/projects/02613_2024/data/celeba/images/
000000 008000 016000 024000 032000 040000 048000 056000 064000 072000
080000 088000 096000 104000 112000 120000 128000 136000 144000 152000
...
$ ls /dtu/projects/02613_2024/data/celeba/images/000000
000000.jpg 000059.jpg 000118.jpg 000177.jpg 000236.jpg 000295.jpg 000354.jpg
000413.jpg 000472.jpg 000531.jpg 000590.jpg 000649.jpg 000708.jpg 000767.jpg
...
```

Issue:

Thousands of files in one folder = slowdowns

Solution:

Make subfolders that each store 1000 files

Pitfall 4: too many files in one folder

```
$ ls bigdata | wc -l
100000
$ time
...
real
```

Remember

```
$ ls /dtu
000000
080000
...
$ ls /dtu
000000.jpg
000413.jpg
...
```

Issue:

```
class zarr.storage.NestedDirectoryStore(path, normalize_keys=False,
dimension_separator: Literal['.', '/'] | None = '/') #
```

[\[source\]](#)

downloads

Storage class using directories and files on a standard file system, with special handling for chunk keys so that chunk files for multidimensional arrays are stored in a nested directory tree.

Notes

The `DirectoryStore` class stores all chunk files for an array together in a single directory. On some file systems, the potentially large number of files in a single directory can cause performance issues. The `NestedDirectoryStore` class provides an alternative where chunk files for multidimensional arrays will be organised into a directory hierarchy, thus reducing the number of files in any one directory.

Safe to write in multiple threads or processes.

0 files

jpg
jpg

Pitfall 5: multi-threading/processing

Two flavors:

1. Too many threads – everyone fighting for cores = overall slowdown
2. Too few threads – not using all available cores = overall slowdown

Pitfall 5: multi-threading/processing

```
#!/bin/bash
#BSUB -J name
#BSUB -q hpc
#BSUB -W 2
#BSUB -R "rusage[mem=512MB]"
#BSUB -n 4
#BSUB -R "span[hosts=1]"
#BSUB -o name_%J.out
#BSUB -e name_%J.err

source /dtu/projects/02613_2024/conda/conda_init.sh
conda activate 02613

python -u script.py input.npy 4
```

CPU model may have 32 cores, but we only get 4

Issue:

Too many threads.

Common causes:

1. Most packages do not respect actual core count (#BSUB -n option)

Pitfall 5: multi-threading/processing

```
#!/bin/bash
#BSUB -J name
#BSUB -q hpc
#BSUB -W 2
#BSUB -R "rusage[mem=512MB]"
```

Issue:

Too many threads.

**This includes Python's multiprocessing!
multiprocessing.Pool() only checks hardware**

```
source /dtu/projects/02613_2024/conda/conda_init.sh
conda activate 02613
```

```
python -u script.py input.npy 4
```

Pitfall 5: multi-threading/processing

```
#!/bin/bash
#BSUB -J name
#BSUB -q hpc
#BSUB -W 2
#BSUB -R "rusage[mem=512MB]"
#BSUB -n 4
#BSUB -R "span[hosts=1]"
#BSUB -o name_%J.out
#BSUB -e name_%J.err

source /dtu/projects/02613_2024/conda/conda_init.sh
conda activate 02613

python -u script.py input.npy 4
```



Starts multiple threads /
processes that each use
multi-threaded package

Issue:

Too many threads.

Common causes:

1. Most packages does not respect actual core count (#BSUB -n option)
2. You are already doing multi-threading/processing in your code.

Pitfall 5: multi-threading/processing

```
#!/bin/bash
#BSUB -J name
#BSUB -q hpc
#BSUB -W 2
#BSUB -R "rusage[mem=512MB]"
#BSUB -n 4
#BSUB -R "span[hosts=1]"
#BSUB -o name_%J.out
#BSUB -e name_%J.err

source /dtu/projects/02613_2024/conda/conda_init.sh
conda activate 02613

OMP_NUM_THREADS=1 # ...or other var or in code
python -u script.py input.npy 4
```

Issue:

Too many threads.

Common causes:

1. Most packages does not respect actual core count (#BSUB -n option)
2. You are already doing multi-threading/processing in your code.

Solution:

1. Disable multi-threading in package
2. Remove your own multi-threading

Pitfall 5: multi-threading/processing

```
#!/bin/bash
#BSUB -J name
#BSUB -q hpc
#BSUB -W 2
#BSUB -R "rusage[mem=512MB]"
#BSUB
#BS
#BS
#BS
```

```
source /dtu/projects/02613_2024/conda/conda_init.sh
conda activate 02613
```

```
OMP_NUM_THREADS=1 # ...or other var or in code
python -u script.py input.npy 4
```

Issue:

Too many threads.

Common

**Moral: be careful with parallel code.
Check your packages!**

respect
(option)

2. You are already doing multi-threading/processing in your code.

Solution:

1. Disable multi-threading in package
2. Remove your own multi-threading

Pitfall 5: multi-threading/processing

```
#!/bin/bash
#BSUB -J name
#BSUB -q hpc
#BSUB -W 2
#BSUB -R "rusage[mem=512MB]"
#BSUB -n 4
#BSUB -R "span[hosts=1]"
#BSUB -o name_%J.out
#BSUB -e name_%J.err

source /dtu/projects/02613_2024/conda/conda_init.sh
conda activate 02613

OMP_NUM_THREADS=4 # ...or other var or in code
python -u script.py input.npy 4
```

Issue:

Too few threads.

Common causes:

Some packages support multi-threading, but it must be turned on

Solution:

Turn it on!

Pitfall 5: multi-threading/processing

Example: NumPy

```
from time import perf_counter as time
import numpy as np
```

```
A = np.random.rand(4000, 4000)
B = np.random.rand(4000, 4000)
```

```
t0 = time()
C = np.matmul(A, B)
t1 = time()
print(t1 - t0, 'seconds')
```

mm.py

```
source /dtu/projects/02613_2024/conda/conda_init.sh
conda activate 02613
```

```
python mm.py
```

run_mm.sh

```
$ sh run_mm.sh
1.723987553967163 seconds
```

Pitfall 5: multi-threading/processing

Example: NumPy

```
from time import perf_counter as time
import numpy as np
```

```
A = np.random.rand(4000, 4000)
B = np.random.rand(4000, 4000)
```

```
t0 = time()
C = np.matmul(A, B)
t1 = time()
print(t1 - t0, 'seconds')
```

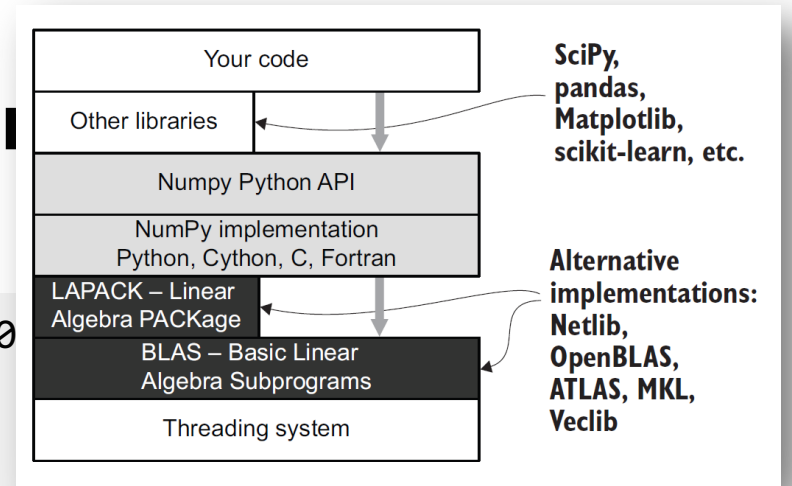
mm.py

```
source /dtu/projects/0
conda activate 02613
```

```
python mm.py
```

run_mm.sh

```
$ sh run_mm.sh
1.723987553967163 seconds
```



Pitfall 5: multi-threading/processing

Example: NumPy

```
from time import perf_counter as time
import numpy as np
```

```
A = np.random.rand(4000, 4000)
B = np.random.rand(4000, 4000)
```

```
t0 = time()
C = np.matmul(A, B)
t1 = time()
print(t1 - t0, 'seconds')
```

mm.py

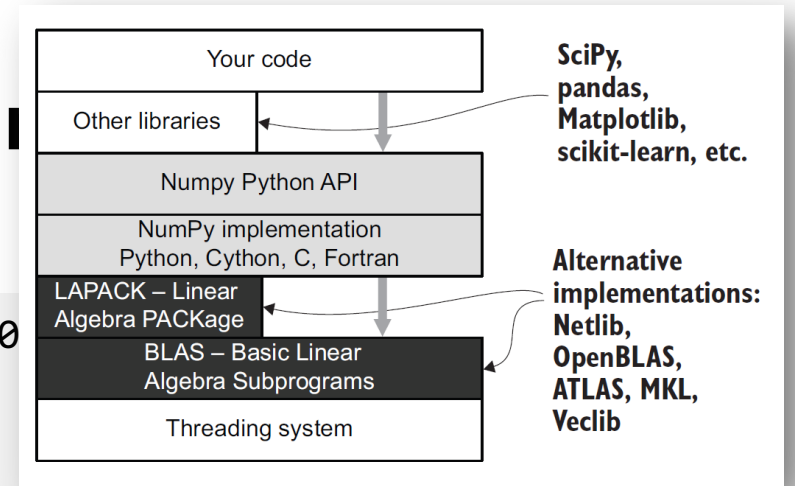
```
source /dtu/projects/0
conda activate 02613
```

```
OMP_NUM_THREADS=8
MPI_NUM_THREADS=8
MKL_NUM_THREADS=8
OPENBLAS_NUM_THREADS=8
```

```
python mm.py
```

run_mm.sh

```
$ sh run_mm.sh
1.723987553967163 seconds
```



Exact variable depends on NumPy backend.
Easiest: set them all

Pitfall 5: multi-threading/processing

Example: NumPy

```
from time import perf_counter as time
import numpy as np
```

```
A = np.random.rand(4000, 4000)
B = np.random.rand(4000, 4000)
```

```
t0 = time()
C = np.matmul(A, B)
t1 = time()
print(t1 - t0, 'seconds')
```

mm.py

```
source /dtu/projects/0
conda activate 02613
```

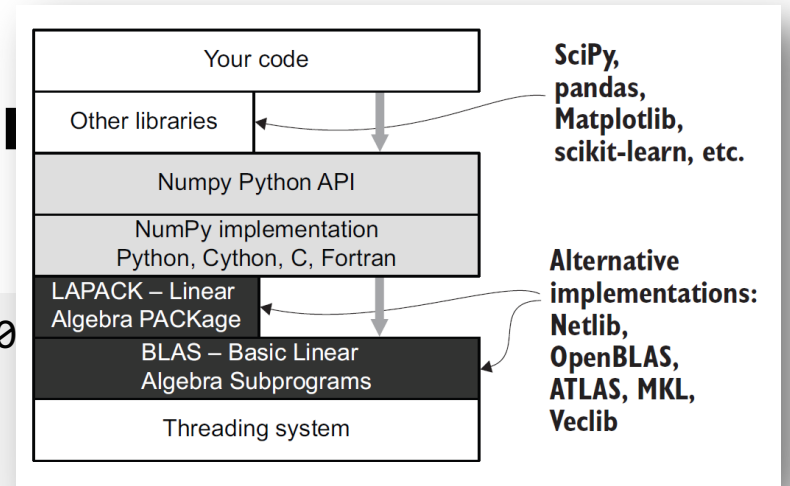
```
NUM_THREADS=8
OMP_NUM_THREADS=$NUM_THREADS
MPI_NUM_THREADS=$NUM_THREADS
MKL_NUM_THREADS=$NUM_THREADS
OPENBLAS_NUM_THREADS=$NUM_THREADS
```

```
python mm.py
```

run_mm.sh

```
$ sh run_mm.sh
1.723987553967163 seconds
$ sh run_mm.sh
0.2338214188348502 seconds
```

7.4x speed-up – almost perfect!



Exact variable depends on NumPy backend.
Easiest: set them all

Pitfall 5: multi-threading/processing

The screenshot shows the SuperFastPython website. The header includes the site name, a tagline, and navigation links. The article title is 'Which NumPy Functions Are Multithreaded'. A red callout box with a white background and a red border contains the text: 'Moral: be careful with parallel code. Check your packages!'. Below the callout, the article text begins with 'In this tutorial, you will discover which NumPy functions support parallelism via multithreading in Python. Let's get started.' On the right, there is a bio for Jason Brownlee and social media links.

SuperFastPython
making python developers awesome at concurrency

About Start Here Topics Guides Products [Free Book](#)

Which NumPy Functions Are Multithreaded

APRIL 19, 2023 / 10 MINUTE READ

Last Updated: 10/10/2023

Some of the most common NumPy functions are multithreaded, meaning they can be executed in parallel. This is a great way to speed up your code, but it can also be tricky to get right. In this tutorial, you will discover which NumPy functions support parallelism via multithreading in Python.

Let's get started.

About Jason Brownlee
Hi, my name is Jason Brownlee, Ph.D. and welcome to *SuperFastPython*! My mission is to make Python developers awesome at concurrency. [Learn more.](#)

Join 10,000+ developers:

[Email](#) [Facebook](#) [YouTube](#) [LinkedIn](#) [Twitter](#)

<https://superfastpython.com/multithreaded-numpy-functions/>

Pitfall 6: leaving processes behind

```
#!/bin/bash
#BSUB -J name
#BSUB -q hpc
#BSUB -W 24:00
#BSUB -R "rusage[mem=512MB] span[hosts=1]"
#BSUB -n 4
#BSUB -o name_%J.out
#BSUB -e name_%J.err

source /dtu/projects/02613_2024/conda/conda_init.sh
conda activate 02613

mymonitor > monitor_${LSB_JOBID}.out &

# script takes approx. 1 hour
python -u script.py input.npy 4
```

Idea:

Starting a monitoring process, to collect e.g. performance data, in the background!

Issue:

mymonitor has an endless loop – and then the job will first finish after 24 hours ("-W 24:00")!

Solution:

Make the monitoring smarter, such that it ends when the "real" work is done!

HPC Pitfalls

Treat these as guidelines and warning signs – not hard rules!

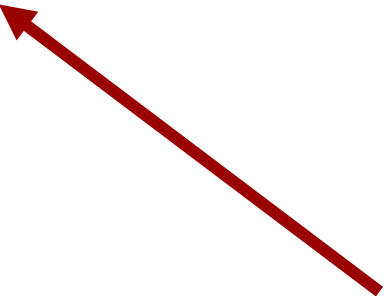
Working on another HPC system? Do some tests – the setup may be different and show other bottlenecks!

Today's exercise

HPC Pitfalls

- Excessive I/O and manual redirection
 - Will only see speed difference on /work3 folders!
 - Test folder for this course: /work3/02613/dump/

- Excessive multi-threading



Use this for -o/-e and output files!
Will be periodically emptied
and later deleted

Useful concepts

Output files for bsub

```
#BSUB -o /work3/02613/dump/name_%J.out
```

```
#BSUB -e /work3/02613/dump/name_%J.err
```

Redirect output

```
python -u script.py \
```

```
1> /work3/02613/dump/output_${LSB_JOBID}.txt \
```

```
2> /work3/02613/dump/error_${LSB_JOBID}.txt
```

NumPy multi-threading

```
NUM_THREADS=8
```

```
OMP_NUM_THREADS=$NUM_THREADS
```

```
MPI_NUM_THREADS=$NUM_THREADS
```

```
MKL_NUM_THREADS=$NUM_THREADS
```

```
OPENBLAS_NUM_THREADS=$NUM_THREADS
```

Dump folder

```
/work3/02613/dump
```

Submit job script

```
bsub < submit.sh
```

Job status

```
bstat / bjobs
```

Check job output

```
bpeek 123456
```

Kill job

```
bkill <JOBID>
```